

## Northwind Home: one agent becomes a team

Domain FDE Pre-Work Deep Track • Blue Data Consulting • Stage 5: Build

### Before you start

Read [usecase1\\_day5\\_problem\\_statement.pdf](#) first: it sets out the business case, the three teams, and how one customer problem crosses customer experience, supply chain and finance. This brief is the practical: how the answer is built in Google ADK, and how you will test it.

The technical question is the one every agent design has to answer: **when a request arrives, who decides the next step, the model or code?** Today you build four answers to it, and prove which one you would ship.

### Today's decision

**How the work is split between agents, how each request finds the right one, and how we prove each split does its job.**

The Day 4 boundary becomes real: “refund” means one thing to complaints and another to Billing, so each team gets its own agent, and each agent only the tools its team owns.

### Three specialists, three tools each

| Specialist | Team         | Its three tools                                           | Rules it owns                                                                  |
|------------|--------------|-----------------------------------------------------------|--------------------------------------------------------------------------------|
| Complaints | CXM          | get_complaint · send_customer_message · escalate_to_human | Vulnerable customers, consent by purpose, no offers                            |
| Returns    | Supply chain | get_return · set_disposition · raise_vendor_claim         | Recalled items only quarantined, grade A to shelf, high value, vendor deadline |
| Billing    | Finance      | get_refunds · issue_refund · request_approval             | Never above the order value; above 25 pounds needs approval                    |

There is no tool that changes an inspector's grade. A tool that does not exist cannot be misused.

### Four ways to put them to work

| # | Agent            | Pattern                        | Next step decided by        | Shows                                      |
|---|------------------|--------------------------------|-----------------------------|--------------------------------------------|
| A | v5a_loop_agent   | Agent loop, Agent approach     | The model                   | One specialist reasoning through its tools |
| B | v5b_loop_graph   | Agent loop, Graph API approach | Code (a graph)              | Draft, check in code, redraft or send      |
| C | v5c_router_graph | Router, Graph API approach     | Code first, then the model  | Each request sent to the right team        |
| D | v5d_orchestrator | Orchestrator, multi-agent      | The model, over specialists | One request, several teams, one answer     |

### What to watch

| Terminal line                 | Meaning                                                                  |
|-------------------------------|--------------------------------------------------------------------------|
| >>> ROUTED BY CODE / BY MODEL | Which team the router chose, and whether code or the model decided       |
| >>> ACTION TAKEN BY AGENT     | A rule allowed the action, and it ran                                    |
| >>> BLOCKED BY GUARDRAIL      | A rule stopped the action. It did not run                                |
| >>> ESCALATED TO HUMAN        | A named person now has it: team lead, Billing lead or returns supervisor |

## What was built

The Day 2 rules, Day 3 recorder and Day 4 ontology now sit in one shared package that every agent uses.

|                             | Day 4                      | Day 5                                                                     |
|-----------------------------|----------------------------|---------------------------------------------------------------------------|
| <b>Agents</b>               | One complaints agent       | Three specialists, used in four patterns                                  |
| <b>Tools</b>                | Six tools in one agent     | Three per specialist, nine in all, each in the team that owns it          |
| <b>Rules</b>                | Five, all about complaints | Nine, owned by three teams, each with a named owner                       |
| <b>Next step decided by</b> | The model, always          | The model in A and D; code in the B and C graphs                          |
| <b>Proof</b>                | Three component tests      | The same tests, plus 7 eval cases, one or two per pattern, and a baseline |

## Your task: 50 minutes, in pairs

**CXM and SCM pairs.** Work in usecase01\_day5. Every command is in README-training-account.md. Record everything in Day5\_Worksheets\_UC01.docx. Do not change the agents; propose changes in ADR-5.

| # | Part            | Time   | What you do                                                                                                        |
|---|-----------------|--------|--------------------------------------------------------------------------------------------------------------------|
| 1 | Predict it cold | 10 min | For each of the 7 eval cases, write which agent handles it and the path you expect.                                |
| 2 | Meet the agents | 10 min | Run the component tests, then look inside each pattern: its tools, or its graph.                                   |
| 3 | Run it          | 15 min | Try each pattern in the chat, including the kettle on the router and the orchestrator. Then run the eval baseline. |
| 4 | Prove it        | 15 min | Classify every failure, choose the pattern you would ship first, and write ADR-5.                                  |

## Seven questions

1. In A the model decides the next step; in B the graph does. What is each better at, and which would you put in front of customers?
2. The router decides by code first. Which requests did the model have to place, and would you trust it with them?
3. A router sends each request to one team. What happened to the kettle, and why is that right for a router?
4. The orchestrator called three specialists. Could it do anything a specialist is not allowed to? How do you know?
5. Each specialist has three tools. Which tool would you add first, and which would you refuse to add?
6. What stops the recalled heater (R-202) going back on the shelf in each pattern: the model or the code?
7. Which eval case would catch a regression in each pattern? Which pattern has no must-never case?

## Bring to the debrief

The pattern you would ship first, and why · the request no pattern handles well · the part of the design you would refuse to sign off.

*All data is made up. Nothing changes in any real system. If the environment breaks, hand up.*